ForgeWeb: Architecture-First Full-Stack Web Application Generation

Pratyush Manas, Rajeev Kumar, Sachin Yash Raj, Sanaa Kumari
Department of Computer Science and Engineering
BMS Institute of Technology and Management, Bengaluru, India
1BY23CS163 1BY23CS171 1BY23CS192 1BY23CS202

***Abstract*—A short product description rarely provides all the decisions needed to build a web application. Generating code immediately can turn missing details into unsuitable layouts, inconsistent interfaces, or incomplete features. ForgeWeb introduces a review step between the initial request and implementation. It prepares requirements and an architecture document, obtains the user's approval, and produces React frontend and Python FastAPI backend artifacts from the agreed specification. The workspace makes generated files, previews, versions, and requirement links available for inspection. Configurable model adapters support the generation stages, while recorded local fallbacks handle selected product domains when a provider is unavailable. This paper explains the prototype's workflow and distinguishes artifact checks from tests of a running application. Fourteen existing automated tests and platform type checking passed in the reproduced evaluation. These results establish evidence for workflow and file-management behavior, rather than a guarantee that every generated application is ready for deployment. The next development priority is isolated execution and requirement-level testing of exported applications.**

***Index Terms*—AI-assisted development, web application generation, architecture approval, language models, requirement traceability.**

I. Introduction

Turning a customer brief into software requires decisions about screens, data, permissions, and interactions. The result must represent the requested product: a restaurant needs a booking journey rather than a renamed dashboard, and an explicit color preference must affect the rendered interface.

Code-generation research demonstrates substantial progress on executable programming tasks. Codex evaluates generated functions through HumanEval [1]; APPS examines a broader collection of programming challenges [2]; and AlphaCode combines generation with selection mechanisms for competition problems [3]. Building an application introduces additional concerns: coordinating several files, matching frontend calls to backend routes, and allowing a person to correct the intended design.

ForgeWeb approaches these concerns through an architecture-first workflow. Its contribution is an inspectable prototype connecting a prompt, an approved specification, generated artifacts, and recorded checks. It does not introduce a new language model. The system keeps ForgeWeb's own interface separate from the customer's requested design and makes provider failures visible instead of treating every output as equivalent.

II. Problem Statement and Objectives

Product descriptions combine features, preferences, and unstated assumptions. Generation may change a name while retaining an unsuitable layout, and an attractive preview may conceal missing server behavior. The problem is to preserve product intent while exposing enough evidence to assess the result. ForgeWeb creates an architecture file before implementation and supports revision, approval, source inspection, and version history. Its objectives are to respect explicit design choices, coordinate frontend and backend contracts, and separate artifact creation from runtime correctness.

III. Related Work

A. Generation and Evaluation

CodeGen explores multi-turn synthesis [4], while InCoder supports generation within existing code context [5]. Both inform the separation of initial construction from later edits. Evaluation research warns against judging output by appearance or a small set of checks. CodeT uses generated tests to select code [6]; EvalPlus strengthens correctness testing [7]; LiveCodeBench considers multiple coding abilities and contamination [8]; and BigCodeBench examines complex instructions involving library functions [9]. These works motivate testing behavior beyond file presence.

B. Agent Workflows and Planning

SWE-bench evaluates repository issue resolution [10], and SWE-agent studies the interface between an agent and its development tools [11]. ChatDev [12], MetaGPT [13], and AutoGen [14] organize collaboration through roles or conversations. Agentless provides a useful counterpoint by examining a simpler repair pipeline [15]. ForgeWeb uses task roles to organize scope, but its current reviewer is a server function rather than an independently reasoning agent.

ReAct connects reasoning to environmental actions [16]. Reflexion [17] and Self-Refine [18] investigate feedback-driven improvement. Chain-of-thought [19] and Plan-and-Solve [20] provide further motivation for intermediate planning. ForgeWeb's architecture document is a user-facing agreement about the product; it is not presented as a reproduction of these reasoning methods.

C. Context, Appearance, and Safety

Retrieval-augmented generation grounds responses in retrieved material [21], and RepoCoder applies retrieval to repository completion [22]. Both suggest future support for edits spanning large projects. Design2Code examines visual frontend reconstruction [23], supporting the need to assess layout separately from syntax. Security studies by Pearce *et al.* [24] and Perry *et al.* [25] show why generated security-related code should not automatically inspire confidence. These findings shape ForgeWeb's validation boundary rather than supply performance claims for the prototype.

IV. System Architecture

A. User Layer and Control Service

The user layer uses React and TypeScript. It contains the prompt composer, proposal review, project drawer, file viewer, preview, and version controls. A Node.js HTTP service coordinates these operations, with Vite middleware exposing the same handler during development. Workflow states distinguish planning, confirmation, generation, review, validation, and completion or failure. Figure 1 outlines this separation.

Project state is persisted in a local JSON store. Queued writes and temporary-file replacement reduce conflicts within the local process. This arrangement supports the prototype, but is not a substitute for a transactional multi-tenant database. Database information shown in the workspace describes intended application entities; it does not prove that a database has been provisioned.

B. Generation and Artifact Storage

In the configured split pipeline, Gemini handles planning and frontend generation, while Groq handles backend generation and OpenRouter can serve as an alternative. This is configurable routing, not an experimentally established ranking of providers. Server-side adapters consume credentials and validate structured responses. The planning output includes product identity, roles, entities, requirements, pages, routes, and design direction.

Local generators cover selected commerce, restaurant, portal, inventory, scheduling, and generic-site cases. They provide a bounded fallback rather than unlimited design originality. Generator mode and failure messages remain attached to the result. Stored artifacts include source, a shared contract, documentation, preview HTML, and file digests. Snapshots support inspection, restoration, and ZIP export.

V. Methodology

A. Planning and Approval

The planner turns a validated prompt into a proposed specification and `ARCHITECTURE.md`. Explicit names and visual preferences should survive normalization. The user can edit the request or confirm the proposal; implementation begins only after confirmation. This boundary checks agreement about scope, not the correctness of later code. Figure 2 shows the sequence and failure branch.

TABLE I
Reproduced Prototype Checks

Test group	Cases	Result
HTTP and approval lifecycle	2	Passed
Domain and visual instructions	4	Passed
Provider fixture and selection	2	Passed
Plan normalization	1	Passed
Workspace, versions, compatibility	3	Passed
Prompt and path rejection	2	Passed
Total	14	Passed
Platform type checking	—	Passed

B. Frontend and Backend Coordination

The approved specification guides React source, styles, FastAPI routes, schemas, access-control files, and supporting tests. A shared JSON contract expresses the intended route and data agreement. Figure 3 separates these files from the preview path. Contract presence reduces ambiguity but cannot establish that implementations agree: a booking form and its endpoint can still interpret dates or identifiers differently.

The embedded preview runs stored HTML in an iframe with script and form permissions. It does not start the generated Python service. Local preview interactions must therefore be distinguished from verified frontend-to-backend requests. Running the exported application requires its dependencies and data services to be configured.

C. Review and Traceability

Planning, implementation, review, and validation records describe objectives, allowed paths, and acceptance criteria. Project policy draws on the supplied Karpathy-derived engineering guidance and gstack workflow material: clarify assumptions, prefer a sufficient solution, limit changes to scope, and inspect evidence. These engineering repositories are not counted as research publications.

The current checks examine paths, manifests, expected artifacts, requirement mappings, and provenance. The internal graph links projects, requirements, tasks, files, and checks. A requirement-to-file edge records an association, not proof that a feature works. Similarly, the graph is not an externally parsed call graph from Code Review Graph. Stronger acceptance would combine the current structural checks with clean builds, API requests, and browser interaction tests against the generated application.

VI. Evaluation and Discussion

The existing platform suite was rerun during paper preparation. All fourteen top-level tests passed, and TypeScript checking completed successfully. Table I groups the observed checks. Tests use temporary stores and deterministic or injected provider results, so they do not measure the quality of live model responses.

The approval checks verify that planning stops for confirmation before implementation artifacts become available. Domain checks inspect product-specific output and explicit

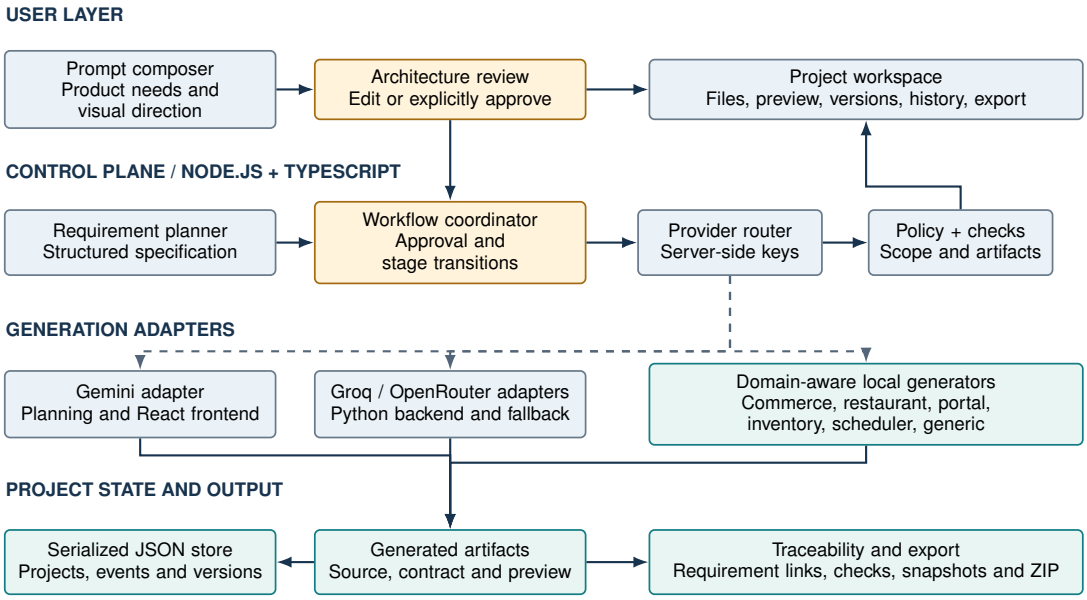


Fig. 1. ForgeWeb's implemented layers. The platform coordinates generation and stores project artifacts; the exported customer application runs separately. Dashed paths represent configurable generation alternatives.

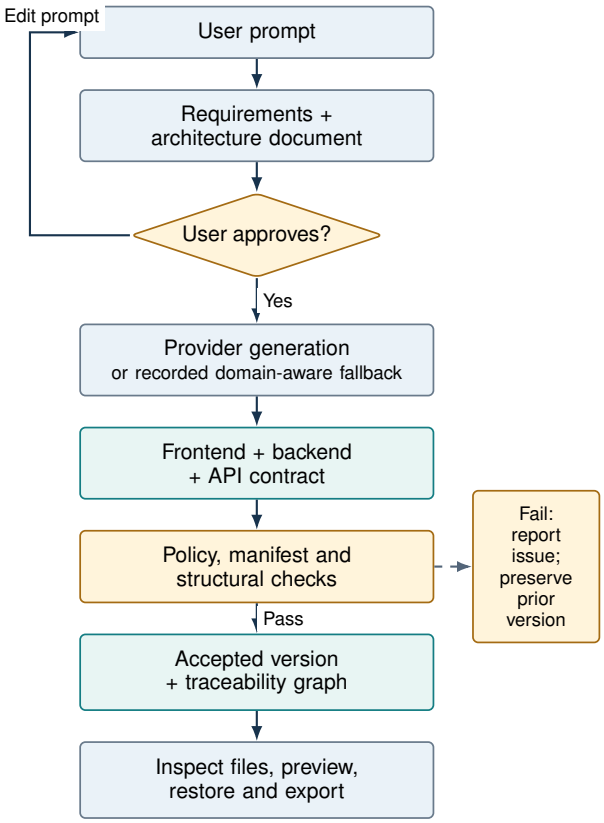


Fig. 2. Data flow from prompt to accepted artifact version. Unaccepted output is not evidence of a successful build.

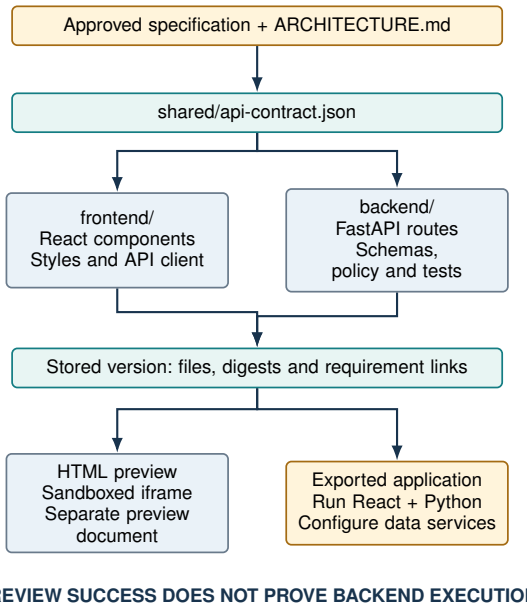


Fig. 3. Generated-system structure. The preview document and the exported application's runtime are separate outputs.

style cues, including a cream-color request. These assertions address earlier repeated-dashboard regressions, but do not measure perceived visual uniqueness. Workspace checks exercise snapshots, restoration, persistence, export, and safe failure. Provider fixtures establish that returned plans and manifests enter the workflow; they do not predict availability under real rate limits.

The results support the orchestration and artifact-management design. They do not establish secure deployment, payment correctness, or successful execution of every generated interaction. A useful next experiment would compare direct generation with architecture-first generation on the same

prompts and provider settings. Measurements should include requirement-level task success, contract agreement, clean-build success, visual adherence, latency, and cost. Such comparative measurements have not been conducted here.

VII. Limitations and Future Work

Source checks can miss broken handlers, incorrect permissions, and mismatched payloads. Fallback designs remain limited, model responses vary, and local persistence is unsuitable for production-scale tenancy.

Future work includes isolated builds, backend execution, persistent application data, and requirement-driven browser tests. Retrieval can support cross-file edits. Additional providers, including OpenAI, require measured evaluation; human assessment must also establish whether layouts meet customer expectations.

VIII. Conclusion

ForgeWeb makes product planning an explicit step before source generation. Its approval boundary, shared specification, inspectable artifacts, and recoverable versions provide a practical basis for reviewing AI-generated applications. The reproduced checks support this workflow while leaving runtime and security claims deliberately limited. Connecting acceptance to executed customer journeys is the next step toward dependable full-stack generation.

References

[1] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[2] D. Hendrycks, S. Basart, S. Kadavath *et al.*, "Measuring coding challenge competence with APPS," *arXiv preprint arXiv:2105.09938*, 2021. [Online]. Available: https://arxiv.org/abs/2105.09938

[3] Y. Li, D. Choi, J. Chung *et al.*, "Competition-level code generation with AlphaCode," *arXiv preprint arXiv:2203.07814*, 2022. [Online]. Available: https://arxiv.org/abs/2203.07814

[4] E. Nijkamp, B. Pang, H. Hayashi *et al.*, "CodeGen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022. [Online]. Available: https://arxiv.org/abs/2203.13474

[5] D. Fried, A. Aghajanyan, J. Lin *et al.*, "InCoder: A generative model for code infilling and synthesis," in *Proc. International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2204.05999

[6] B. Chen, F. Zhang, A. Nguyen *et al.*, "CodeT: Code generation with generated tests," *arXiv preprint arXiv:2207.10397*, 2022. [Online]. Available: https://arxiv.org/abs/2207.10397

[7] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," *arXiv preprint arXiv:2305.01210*, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[8] N. Jain, K. Han, A. Gu *et al.*, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024. [Online]. Available: https://arxiv.org/abs/2403.07974

[9] T. Y. Zhuo, M. C. Vu, J. Chim *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," *arXiv preprint arXiv:2406.15877*, 2024. [Online]. Available: https://arxiv.org/abs/2406.15877

[10] C. E. Jimenez, J. Yang, A. Wettig *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770

[11] J. Yang, C. E. Jimenez, A. Wettig *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[12] C. Qian, W. Liu, H. Liu *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 174–15 186. [Online]. Available: https://aclanthology.org/2024.acl-long.810/

[13] S. Hong, M. Zhuge, J. Chen *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023. [Online]. Available: https://arxiv.org/abs/2308.00352

[14] Q. Wu, G. Bansal, J. Zhang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[15] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024. [Online]. Available: https://arxiv.org/abs/2407.01489

[16] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[17] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *arXiv preprint arXiv:2303.11366*, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[18] A. Madaan, N. Tandon, P. Gupta *et al.*, "Self-Refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[19] J. Wei, X. Wang, D. Schuurmans *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *arXiv preprint arXiv:2201.11903*, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[20] L. Wang, W. Xu, Y. Lan *et al.*, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 2609–2634. [Online]. Available: https://aclanthology.org/2023.acl-long.147/

[21] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *arXiv preprint arXiv:2005.11401*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[22] F. Zhang, B. Chen, Y. Zhang *et al.*, "RepoCoder: Repository-level code completion through iterative retrieval and generation," in *Proc. 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484. [Online]. Available: https://aclanthology.org/2023.emnlp-main.151/

[23] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," in *Proc. 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 3956–3974. [Online]. Available: https://aclanthology.org/2025.naacl-long.199/

[24] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2022. [Online]. Available: https://arxiv.org/abs/2108.09293

[25] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799. [Online]. Available: https://arxiv.org/abs/2211.03622